Programação 2

Aula 8 - Estruturas de Dados: listas, árvores e grafos

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Recapitulando

String: Sequências de Caracteres

Nativamente a linguagem Java criou um tipo de dados novo para suportar sequências de caracteres. O nome deste tipo de dado é **String**.

Assim, em Java, **String é uma classe** e não um tipo primitivo/predefinido.

Operações com **Strings** ocorrem de acordo com o que essa classe possibilita.

String: Sintaxe

Classe

Variável **text** é um objeto, instância da classe String

Texto entre aspas que será atribuído ao objeto no momento seguinte à sua criação

```
String text = "Este é um exemplo de uma sequência de caracteres";
char caractere = text.charAt(3); // qual é esse caractere?
```

Tipo primitivo/ predefinido

Método da classe String que permite recuperar um char em uma posição específica

O valor da variável [caractere] é **'e'**

Saída de Texto no Console (Output)

Classe System que controla aspectos básicos do funcionamento de programas Java

```
String text = "Este é um exemplo de uma sequência de caracteres";
char caractere = text.charAt(3); // qual é esse caractere?

System.out.println(text);
```

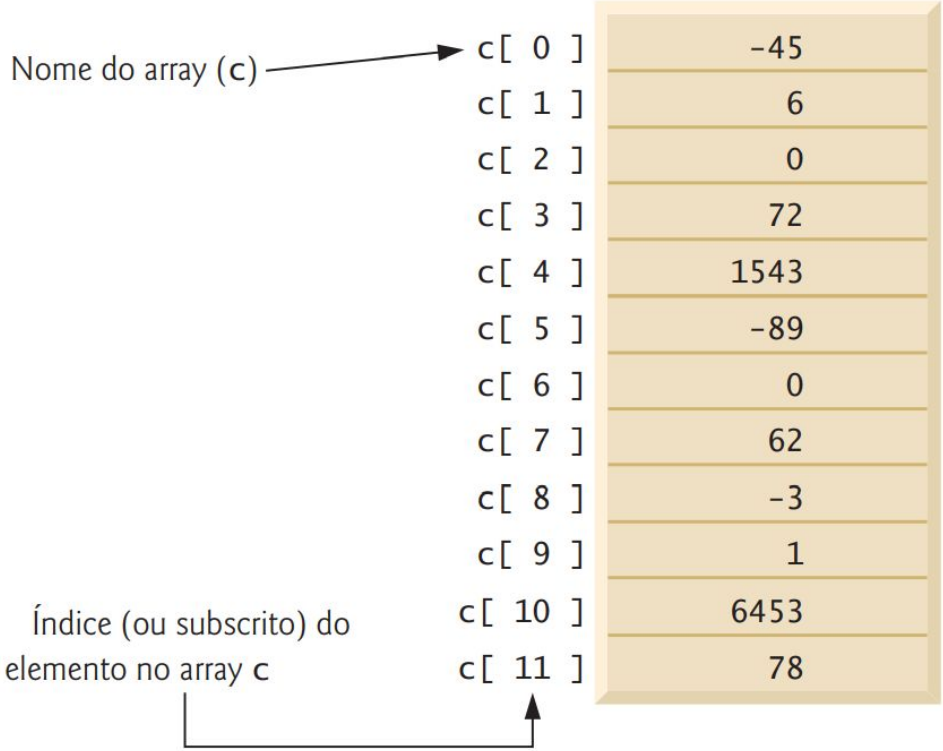
Método println que imprime uma String e insere uma quebra de linha no console

Problems @ Javadoc Declaration Console

```
<terminated> Main [Java Application] C:\Users\Lucas Figueiredo\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_16.0.2.v2
Este é um exemplo de uma sequência de caracteres
```

Console na IDE Eclipse

Arrays: Acesso




```
c[a + b] += 2;
```

O que aconteceu na linha acima?

```
int a = 3;
int b = 1;
c[3 + 1] += 2;
c[4] == 1545 //Essa comparação é true
```

Arrays: Declaração: Sintaxe

Nome do objeto array

Com um número de elementos pré-definido

```
int[] arrayName = new int[21];
```

Tipo dos elementos dentro do array

Criação de um novo array

Arrays: Declaração

Declaração + criação e atribuição na mesma linha

```
int[] c = new int[12];
```

```
String[] b = new String[100]; // cria array b
String[] x = new String[27]; // cria array x
```

Declaração e depois a atribuição

```
int[] c; // declara a variável de array
c = new int[12]; // cria o array; atribui à variável de array
```

Arrays: Declaração e Lista de Inicializador

É possível já **atribuir valores específicos para os elementos** de um array no momento de sua declaração da seguinte forma

```
int[] n = { 10, 20, 30, 40, 50 };
```

Arrays: Iteração em Arrays com a Instrução **for**

No nosso caso inicial, ao usarmos arrays teríamos a mudança a seguir, com os **acessos** começando do **0**

```
// cadastro das notas antes do exercício
float notaAluno01 = 9.0f;
float notaAluno02 = 9.5f;
float notaAluno03 = 8.5f;
//...
float notaAluno21 = 8.3f;


// acréscimo de meio ponto para todos
notaAluno01 += 0.5f;
notaAluno02 += 0.5f;
notaAluno03 += 0.5f;
//...
notaAluno21 += 0.5f;
```

sem array

```
// cadastro das notas antes do exercício
float[] notasAlunos = new float[21];
float notasAlunos[0] = 9.0f;
float notasAlunos[1] = 9.5f;
float notasAlunos[2] = 8.5f;
//...
float notasAlunos[20] = 8.3f;


// acréscimo de meio ponto para todos
for(int i = 0; i < notasAlunos.length; i++)
{
    notasAlunos[i] += 0.5f;
}
```

com array

Matrizes

```
int[][] a = new int[3][4];
```

	Coluna 0	Coluna 1	Coluna 2	Coluna 3
Linha 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Linha 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Linha 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Índice de coluna
Índice de linha
Nome do array

ArrayList: Métodos Disponíveis

Método	Descrição
`add`	Adiciona um elemento ao *final* do `ArrayList`.
`clear`	Remove todos os elementos do `ArrayList`.
`contains`	Retorna `true` se o `ArrayList` contém o elemento especificado; caso contrário, retorna `false`.
`get`	Retorna o elemento no índice especificado.
`indexOf`	Retorna o índice da primeira ocorrência do elemento especificado no `ArrayList`.
`remove`	Sobrecarregado. Remove a primeira ocorrência do valor especificado ou o elemento no índice especificado.
`Size`	Retorna o número de elementos armazenados em `ArrayList`.
`trimToSize`	Corta a capacidade do `ArrayList` para o número atual de elementos.

Figura 7.23 | Alguns métodos e propriedades da classe `ArrayList<T>`.

Conceito de Hoje

Conceito de Hoje

Estruturas de Dados

Listas Ligadas,
Árvores e
Grafos

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Introdução

Problema

Motivação

História e Pessoas

Como lidar com **muitos dados** do **mesmo tipo**?

Digamos que eu queira monitorar a nota de todos da turma.

Em determinado exercício que fizemos em sala, foi definido que **a nota de todos os 21 alunos da turma aumentaria em 0.5**.

Como faríamos isso em código?

21 DISCENTES FORAM MATRICULADOS NESSA TURMA

Matrícula	Nome	Curso	Situação
200721801	ALYSSON COSTA PEREIRA	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721814	CESAR YURI E SILVA SANTOS	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721804	ELIAS VITOR DA SILVA	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721803	ENOCK DE MATOS LEAL NETO	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721824	EVERSON EDUARDO DA SILVA ANDRADE	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200723884	FABIO LUIZ DA SILVA FILHO	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721805	FELIPE BERTULINO DA SILVA	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721797	FELIPE MUNIZ GARCEZ	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721817	HELSON GONCALVES DOS SANTOS FILHO	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721807	ITALO RYAN PIMENTEL HORAS	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721816	JAMERSON CARLOS DOS SANTOS	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200723886	JOANA BEATRIZ SILVA BRANDAO	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721820	JOAO BATISTA MENDES DA SILVA FILHO	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721802	JOSE ALLEX DA SILVA MOURA	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200723865	JOSE MATEUS DE LIMA CHAGAS	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO

Simulando uma fila com arrays

Para tirar a pessoa da frente temos que mover todos os elementos do array o que custa **N operações** (movendo todos elementos em 1 posição)

```java
public class Pessoa {
    String nome;
    int cpf;
    String cor;
    float altura;
    Data dataNascimento;

    Pessoa(String nome)
    {
        this.nome = nome;
    }
}
```

```java
Run | Debug
public static void main(String[] args){
    Pessoa[] fila = new Pessoa[10];

    fila[0] = new Pessoa(nome: "João");
    fila[1] = new Pessoa(nome: "Gleyson");
    fila[2] = new Pessoa(nome: "Fernando");
    fila[3] = new Pessoa(nome: "Tales");
    fila[4] = new Pessoa(nome: "Marcos");

    // joão foi atendido

    for (int i = 0; i < fila.length -1; i++) {
        fila[i] = fila[i+1];
        System.out.println(fila[i].nome);
    }
}
```

Apresentação

Definições

Exemplos Ilustrativos

Listas Ligadas (*Linked Lists*)

```
1
2 public class Person {
3     public Person(String name, int age)
4     {
5         this.name = name;
6         this.age = age;
7     }
8
9     public String name;
10    public int age;
11    public Person next;
12 }
```

o que a **linha 11** significa?

o que possibilita?

Listas Ligadas (*Linked Lists*)

Outro conceito para **conectar elementos de um mesmo tipo** é o de Listas Ligadas

Listas Ligadas é uma **forma mais dinâmica de representar coleções** de elementos, facilitando por exemplo a

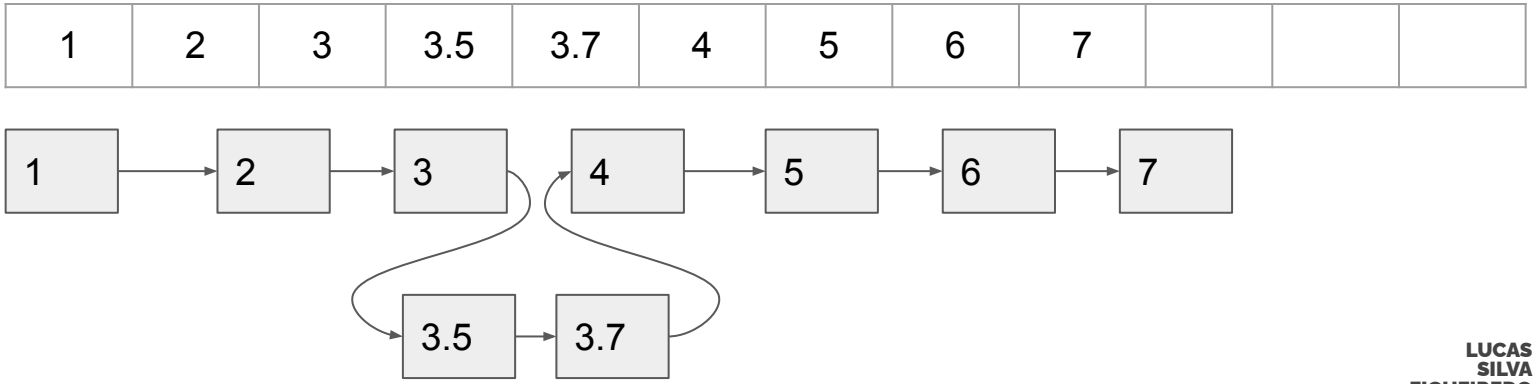
adição de um elemento na posição do meio da lista

Listas Ligadas (*Linked Lists*)

Outro conceito para **conectar elementos de um mesmo tipo** é o de Listas Ligadas

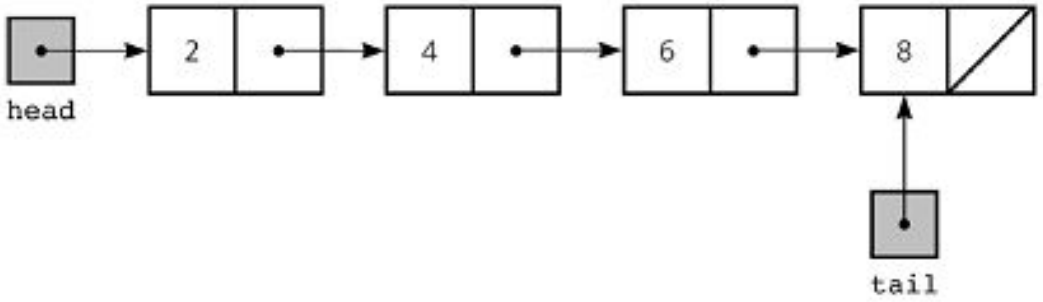
Listas Ligadas é uma **forma mais dinâmica de representar coleções** de elementos, facilitando por exemplo a

adição de um elemento na posição do meio da lista



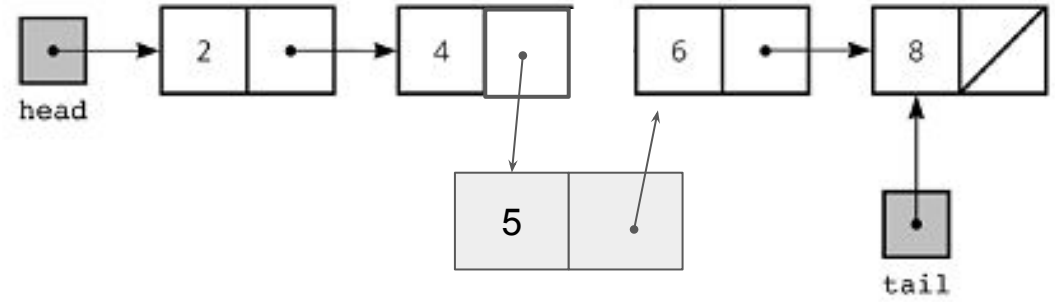
Listas Ligadas (*Linked Lists*)

A imagem abaixo ilustra uma lista ligada.



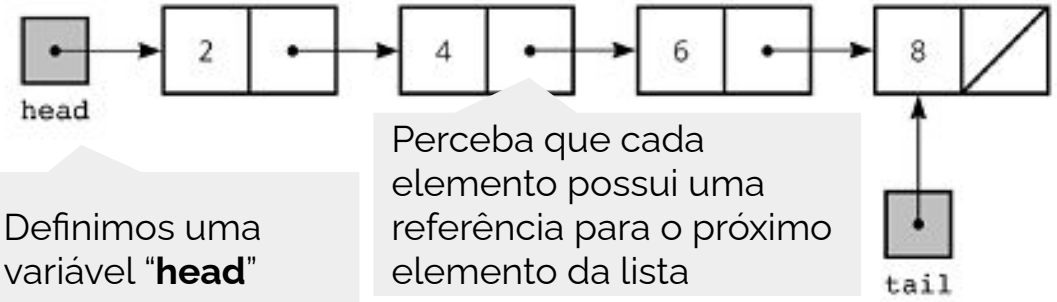
Listas Ligadas (*Linked Lists*)

A imagem abaixo ilustra uma lista ligada.



Listas Ligadas (*Linked Lists*)

A imagem abaixo ilustra uma lista ligada.



Definimos uma variável "**head**" para acessarmos o início da lista

Perceba que cada elemento possui uma referência para o próximo elemento da lista

Geralmente se define uma variável "**tail**" para facilitar o acesso ao fim da lista

```
1
2 public class Person {
3     public Person(String name, int age)
4     {
5         this.name = name;
6         this.age = age;
7     }
8
9     public String name;
10    public int age;
11    public Person next;
12 }
```

```
Person head = new Person("Ana", 22);
head.next = new Person("Maria", 21);
head.next.next = new Person("Paulo", 18);
head.next.next.next = new Person("Bianca", 37);
```

Listas Ligadas (no código)

Para uma fila de atendimento

```java
public class Pessoa {
    String nome;
    int cpf;
    String cor;
    float altura;
    Data dataNascimento;
    Pessoa proxima;

    Pessoa(String nome)
    {
        this.nome = nome;
    }
}
```

```java
private static void mostrarFila(Pessoa cabeca) {
    Pessoa atual = cabeca;
    while(atual != null)
    {
        System.out.println(atual.nome);
        atual = atual.proxima;
    }
}
```

```java
Run | Debug
public static void main(String[] args){
    Pessoa joao     = new Pessoa(nome: "João");
    Pessoa gleyson  = new Pessoa(nome: "Gleyson");
    Pessoa fernando = new Pessoa(nome: "Fernando");
    Pessoa tales    = new Pessoa(nome: "Tales");
    Pessoa marcos   = new Pessoa(nome: "Marcos");
    joao.proxima = gleyson;
    gleyson.proxima = fernando;
    fernando.proxima = tales;
    tales.proxima = marcos;

    Pessoa cabeca = joao;
    mostrarFila(cabeca);

    System.out.println(x: "---joão foi atendido e lucas furou---");

    cabeca = joao.proxima;
    Pessoa lucas   = new Pessoa(nome: "Lucas");
    lucas.proxima = fernando.proxima;
    fernando.proxima = lucas;
    mostrarFila(cabeca);

}
```

Para tirar a pessoa
da frente da fila basta
1 operação
(no caso, mudar a “cabeça” da fila)

Árvores

Árvores são uma generalização de Listas Ligadas com Bifurcações (ou Trifurcações, ou ...)

Nas árvores existe um elemento normalmente chamado de **raiz** que é análogo ao que a variável **head** atende no conceito de listas

Árvores

Ao lado segue um exemplo de árvore.

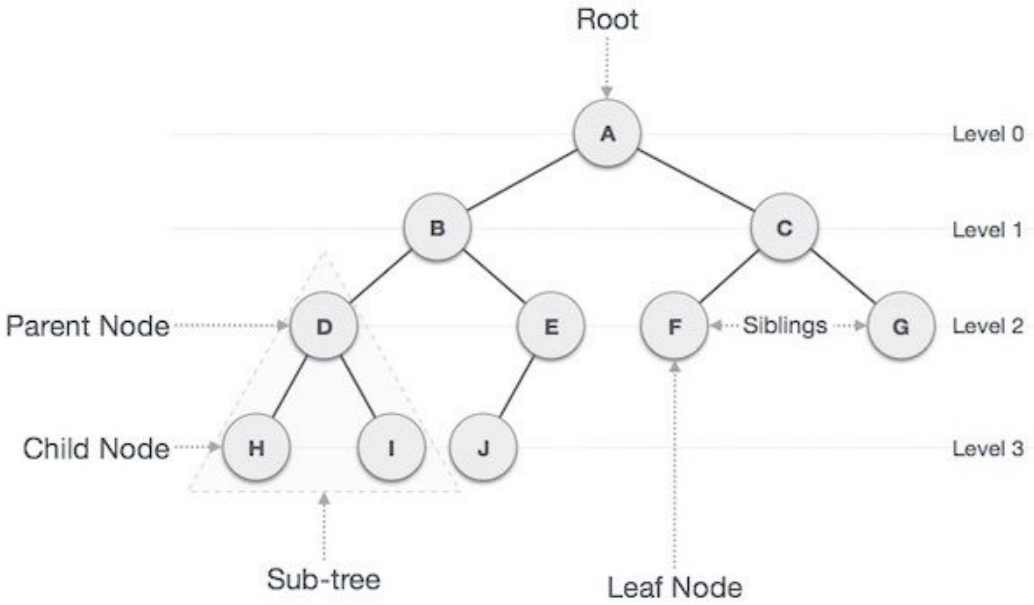
Root denota a raiz da árvore.

Cada elemento é comumente chamado de nó (**Node**).

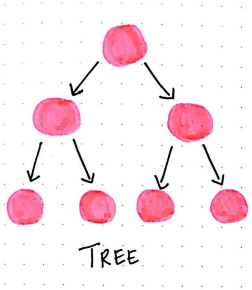
Existe uma hierarquia evidente, nomeando as relações entre os nós como relações de parentescos

Parent Node,
Child Node,
Siblings

Os nós que não tem filhos são comumente chamados de folhas (**Leaf**)



Árvores (no código)



Uma árvore genealógica

```java
public class Pessoa {
    String nome;
    int cpf;
    String cor;
    float altura;
    Data dataNascimento;
    Pessoa mae;
    Pessoa pai;

    Pessoa(String nome)
    {
        this.nome = nome;
    }
}
```

```java
public static void mostrarArvore(Pessoa raiz)
{
    Pessoa atual = raiz;
    if(atual.mae != null && atual.pai != null)
    {
        System.out.println(x: " --- ");
        System.out.println(atual.nome + " é filho de:");
        System.out.println(atual.mae.nome + " e ");
        System.out.println(atual.pai.nome);
        System.out.println(x: " --- ");
        mostrarArvore(atual.mae);
        mostrarArvore(atual.pai);
    }
}
```

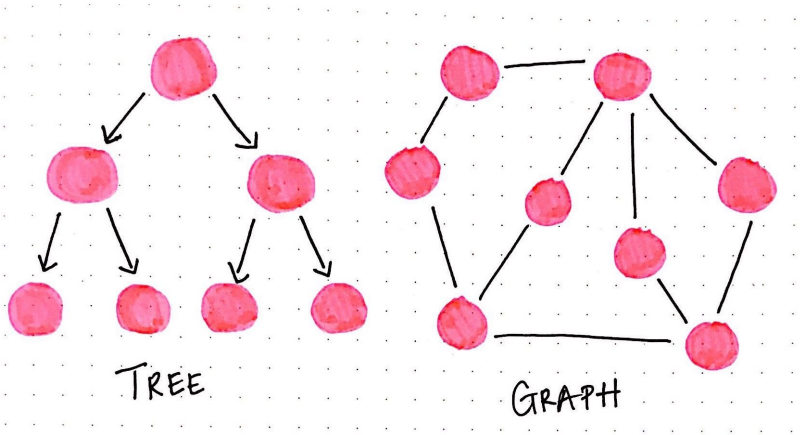
```java
Run | Debug
public static void main(String[] args){
    Pessoa matheus  = new Pessoa(nome: "Matheus");
    Pessoa arnaldo  = new Pessoa(nome: "Arnaldo");
    Pessoa luciana  = new Pessoa(nome: "Luciana");
    Pessoa armando  = new Pessoa(nome: "Armando");
    Pessoa ilka     = new Pessoa(nome: "Ilka");
    Pessoa jose     = new Pessoa(nome: "Jose");
    Pessoa maria    = new Pessoa(nome: "Maria");
    matheus.pai = arnaldo;
    matheus.mae = luciana;
    matheus.pai.pai = armando;
    matheus.pai.mae = ilka;
    matheus.mae.pai = jose;
    matheus.mae.mae = maria;

    Pessoa raiz = matheus;
    mostrarArvore(raiz);
}
```

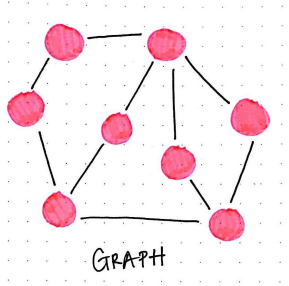
Grafos

Grafos por sua vez são a generalização das árvores. Em grafos não existe a ideia de hierarquia

Ao lado segue uma imagem comparando o exemplo de uma árvore com um grafo



Grafos (no código)



E se fizéssemos uma rede social (ex.: instagram)

```java
public class Pessoa {
    String nome;
    int cpf;
    String cor;
    float altura;
    Data dataNascimento;
    ArrayList<Pessoa> seguindo;
    boolean mostrado;

    Pessoa(String nome)
    {
        this.mostrado = false;
        this.nome = nome;
        seguindo = new ArrayList<Pessoa>();
    }
}
```

```java
public static void mostrarGrafo(Pessoa no)
{
    if(no.mostrado == false)
    {
        Pessoa atual = no;
        System.out.println(x: " --- ");
        System.out.println(atual.nome + " está seguindo:");
        for (Pessoa p : atual.seguindo)
        {
            System.out.println(p.nome);
        }

        atual.mostrado = true;

        for (Pessoa p : atual.seguindo)
        {
            mostrarGrafo(p);
        }
    }
}
```

```java
Run | Debug
public static void main(String[] args){
    Pessoa matheus  = new Pessoa(nome: "Matheus");
    Pessoa arnaldo  = new Pessoa(nome: "Arnaldo");
    Pessoa gleyson  = new Pessoa(nome: "Gleyson");
    Pessoa galvao   = new Pessoa(nome: "Galvão");
    Pessoa tales    = new Pessoa(nome: "Tales");
    matheus.seguindo.add(gleyson);
    gleyson.seguindo.add(tales);
    gleyson.seguindo.add(arnaldo);
    gleyson.seguindo.add(galvao);
    arnaldo.seguindo.add(matheus);
    arnaldo.seguindo.add(galvao);
    galvao.seguindo.add(arnaldo);

    mostrarGrafo(gleyson);
}
```

Exercício

Mini-Projeto 5

Criar uma **classe Escolha** para substituir as strings utilizadas. Cada escolha deve ter pelo menos dois atributos: **String** texto; **Capitulo** proximo.

O construtor da classe Escolha deve receber o texto e o objeto que representa o próximo capítulo derivado daquela escolha.

O construtor da classe Capitulo não deverá mais receber as escolhas como parâmetros

Ao invés disso, todos os capítulos serão criados sem informar as escolhas primeiramente

Na sequência, as escolhas devem ser adicionadas aos capítulos

Um objeto Capitulo raiz deve ser criado referenciando o objeto do capítulo inicial

O método **mostrar()** deve chamar o método escolher dentro. Em seguida o deve chamar o método mostrar do capítulo escolhido (que está armazenado dentro do objeto Escolha)

No método main o único método chamado dos capítulos deve ser o método mostrar do capítulo raiz:

raiz.mostrar(); // pois o capítulo raiz vai chamar o método mostrar dos capítulos escolhidos

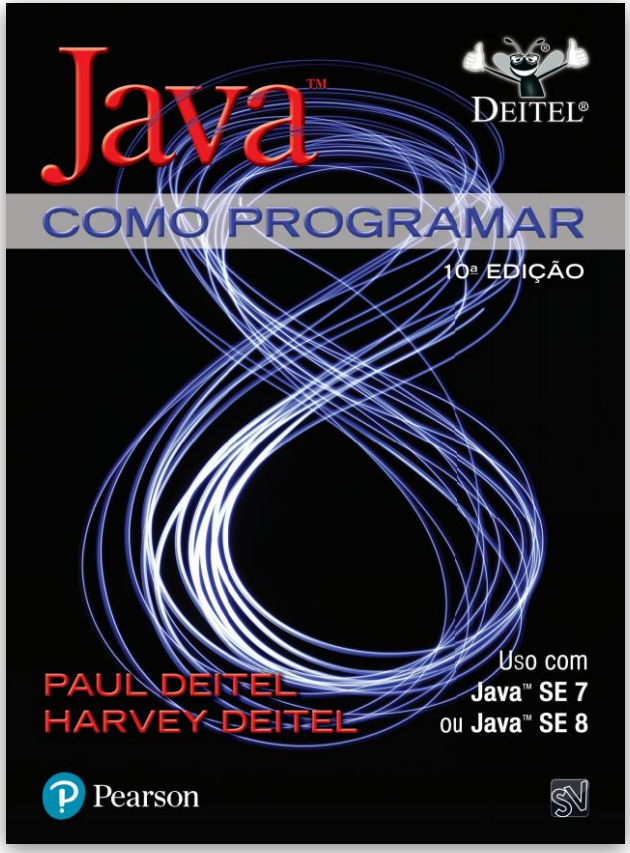
Referências

Java - Como Programar (10a Edição)

7 Arrays e ArrayLists

21.4 Listas encadeadas

21.7 Árvores

Ferramenta:

Eclipse: eclipse.org/downloads